Exhaustive Threat Intelligence Report: Static Differential Analysis and Historical Profiling of PyPI Supply Chain Attacks

1. Executive Summary and Strategic Context

The software supply chain has evolved into the primary vector for advanced persistent threats (APTs) and financially motivated cybercriminals. As the central repository for the Python ecosystem, the Python Package Index (PyPI) manages an ever-expanding infrastructure that serves as the foundation for global enterprises, academic institutions, and government agencies. Historically, the registry model was built on implicit trust, unrestricted publishing, and dynamic, complex cross-dependencies. This architecture, while fostering rapid open-source innovation, has inadvertently created a highly leveraged attack surface. The paradigm of cyber warfare has shifted; instead of targeting hardened organizational endpoints or enterprise perimeters, threat actors now target the developer environments and build pipelines that ultimately generate the software consumed by those enterprises.

By injecting malicious code upstream—whether through typographical mimicking, dependency confusion, or the outright compromise of trusted maintainer accounts—attackers achieve a one-to-many infection ratio. The deployment of a single poisoned package can automatically propagate to thousands of downstream consumers via automated dependency resolution tools like pip. The development of the Python Package Index Supply Chain Attack Detection Analysis (PyPI-SCADA) simulator represents a critical advancement in defensive methodologies. By utilizing a local dummy Simple API server that adheres to PEP 503 standards to perform static differential analysis (diffing) between consecutive package versions, defenders can computationally isolate logical anomalies introduced during a package update. However, to calibrate such a sophisticated simulator effectively, a highly precise historical baseline of known attacks must be established. A detection engine is only as robust as the heuristic baseline upon which it is trained. This report provides an exhaustive forensic analysis of specific historical supply chain attacks, charting their mechanisms, timelines, code execution triggers, and broader ecosystem implications to optimize the detection algorithms of the PyPI-SCADA engine. The analysis spans multiple eras of package registry exploitation, from rudimentary typosquatting campaigns in 2018 to highly sophisticated, multi-stage remote access trojans and continuous integration pipeline hijacks observed in 2024 and 2025.

2. The Architectural Vulnerabilities of the Python Ecosystem

To understand how these specific packages compromised their targets, one must first examine the underlying mechanics of the Python packaging ecosystem. Python's design philosophy

prioritizes developer flexibility and ease of deployment, which inadvertently exposes several structural vulnerabilities that malicious actors exploit.

2.1 The Execution of Arbitrary Code During Installation

Unlike compiled languages where dependencies are securely linked during a controlled build process, Python packages frequently execute arbitrary code during the installation phase itself. When a developer or an automated system runs pip install, the package manager queries the PyPI index, downloads the appropriate distribution, and attempts to install it. If a pre-compiled binary wheel (.whl) is not available or compatible, pip downloads the source distribution (.tar.gz or .zip) and executes the setup.py file to build the package locally.[1]

This execution of setup.py happens with the privileges of the user running the pip command. Threat actors have long recognized this as the most reliable trigger for malware execution. By embedding malicious Python code within the setup() function—or by overriding standard setup commands like install or build_ext—an attacker guarantees that their payload runs immediately upon package download, long before the developer ever imports the library into their project.[3] The PyPI-SCADA simulator must therefore prioritize the diffing of setup scripts and configuration files (pyproject.toml, setup.cfg) as these are the primary staging grounds for initial infection vectors.

2.2 The Simple Repository API (PEP 503) and Index Prioritization

The PyPI-SCADA system relies on simulating a local PEP 503 Simple API server. PEP 503 defines the standard interface for Python package repositories, relying on simple HTML documents containing anchor tags (<a href="...">) that link to package distribution files. When an organization uses internal, proprietary packages, they often host them on a private PEP 503 compliant server. To tell pip to look for packages on this private server while still accessing public packages from PyPI, developers frequently use the --extra-index-url configuration.

This configuration is structurally flawed. When pip encounters an --extra-index-url, it does not prioritize the private index over the public one. Instead, it queries both indices, combines the results for a given package name, and installs the package with the highest semantic version number, regardless of which index it originated from.[5] This exact mechanism enables dependency confusion attacks, wherein an attacker registers the name of an internal package on the public PyPI repository and assigns it an artificially high version number, ensuring the package manager pulls the malicious public payload instead of the legitimate internal library.[6]

2.3 Automated Dependency Resolution

The final structural vulnerability is the modern reliance on automated dependency management. Tools like Dependabot or Renovate are designed to keep software secure by automatically generating pull requests to update dependencies to their latest versions. However, in the event of an account takeover where a legitimate package is poisoned (Fruit Poisoning), these automated tools act as super-spreaders. They immediately detect the new, malicious version on PyPI and push it into enterprise continuous integration and continuous deployment (CI/CD) pipelines before human security teams have time to react.[8] This

paradox—where automation designed for security accelerates malware propagation—necessitates the kind of rapid, automated static differential analysis that PyPI-SCADA aims to provide.

3. The Theoretical Framework of PyPI-SCADA

The detection framework for which this forensic data is being prepared relies on static differential analysis, a technique that moves beyond simplistic static application security testing (SAST) by establishing a temporal and comparative baseline.

3.1 The Limitations of Traditional SAST in Supply Chain Contexts

Traditional SAST tools evaluate a codebase in isolation, scanning for known malicious signatures, insecure function calls (such as eval(), exec(), or subprocess.Popen()), or hardcoded cryptographic secrets. While highly effective for identifying accidental vulnerabilities or poor coding practices, traditional SAST struggles profoundly against intentional, hostile supply chain poisoning.

Attackers employ sophisticated obfuscation techniques—ranging from Base64 and hexadecimal encoding to dynamic payload generation at runtime—that easily bypass static signature matching.[1] Furthermore, because Python is an inherently dynamic language used heavily in systems administration and infrastructure automation, legitimate packages frequently execute shell commands, manipulate file systems, and establish network connections. This reality leads to an unmanageable false-positive rate for traditional SAST tools when applied blindly to an entire package index. Flagging every package that imports the os or socket library would render the detection system useless due to alert fatigue.

3.2 The Mechanics of Static Differential Analysis

Differential analysis mitigates the false-positive dilemma by introducing a temporal comparative dimension. Instead of analyzing a package in a vacuum, the PyPI-SCADA engine evaluates the delta—the exact logical diff—between the Last Known Good Release (LKGR) and the newly uploaded version.

When an attacker compromises a legitimate package, they typically inject a highly concentrated, obfuscated payload into an otherwise benign update. By extracting the Abstract Syntax Trees (AST) of both the LKGR and the new version, and computationally comparing them, the simulator can isolate the exact logical modifications. If a popular mathematics library that has historically only performed floating-point calculations suddenly introduces an update that imports the urllib, os, and base64 libraries to execute a hidden string, the differential engine flags a high-severity anomaly.[4] The system does not need to know the specific signature of the malware; the profound deviation from the package's historical behavioral baseline is sufficient for detection.

For attack vectors like typosquatting and dependency confusion, differential analysis encounters a different paradigm. These packages do not have a benign predecessor; their version history begins with malware. In these scenarios, the PyPI-SCADA simulator must pivot from intra-package diffing to ecosystem-wide metadata diffing. This involves comparing the

new package's name against the Top 10,000 most downloaded PyPI packages using Levenshtein distance algorithms to detect typosquatting, and employing deep heuristic analysis of the setup.py file to detect immediate out-of-bounds network requests or file system traversal.[10]

4. The Backstabber's Knife Collection: Establishing a Baseline

The foundational dataset for understanding and calibrating supply chain detection models is the "Backstabber's Knife Collection" (BKC).[11] First presented by Ohm et al. in 2020 at the Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA) conference, this dataset meticulously curated malicious open-source software components observed in real-world attacks dating back to 2015 across npm, PyPI, and RubyGems.[9]

The BKC is vital for the PyPI-SCADA project because it systematically categorizes attacks by their injection technique, primary objective, targeted operating system, and execution trigger.[9] The initial analysis of the dataset revealed that malicious PyPI packages historically favored execution during the installation phase, aggressively leveraging the setup.py vulnerability.[3] The primary objective in the vast majority of these early cases was explicit data exfiltration—specifically targeting developer credentials, .npmrc files, SSH keys, and environmental variables that could lead to broader infrastructure compromise.[3]

As the Python ecosystem has matured and grown in economic significance, the sophistication of these attacks has escalated exponentially. The threat landscape has transitioned from simple typosquats created by opportunistic script kiddies for minor cryptocurrency theft, to highly orchestrated campaigns executed by sophisticated threat actors targeting specific artificial intelligence, machine learning, and military-grade CI/CD workflows. The BKC serves as the historical anchor, providing the raw package artifacts necessary to test whether a modern differential analysis engine can retroactively catch the exploits that previously compromised the ecosystem.[12]

5. Exhaustive Forensic Analysis of Target Attack Vectors

To effectively calibrate the PyPI-SCADA simulator, a precise historical timeline and technical reconstruction must be established for the specific packages requested. The following sections provide a deep forensic breakdown of each target package, correcting discrepancies in assumed version numbers where necessary, and detailing the specific mechanisms of compromise, infection dates, and Last Known Good Releases (LKGR).

5.1 Dependency Confusion Attacks

Dependency confusion remains one of the most elegant and structurally devastating attacks in the supply chain arsenal. It relies entirely on the predictable, default behavior of package managers prioritizing public registries over private ones when namespace collisions occur.[5]

5.1.1 The torchtriton Compromise

The torchtriton incident is widely regarded as one of the most prominent dependency confusion attacks in PyPI history, primarily due to its direct targeting of the high-value artificial intelligence and machine learning ecosystem. PyTorch, a premier open-source machine learning framework developed by Meta AI, utilized an internal dependency named torchtriton for compiling its nightly Linux builds.[6] This internal dependency was a specialized extension of the PyTorch Triton library, allowing users to leverage models trained in PyTorch within TensorRT inference accelerators.[7] Crucially, this package was hosted exclusively on PyTorch's private, dedicated package index and had never been registered on the public PyPI database.[6]

On December 25, 2022, an attacker operating under the guise of an ethical security researcher registered the exact name torchtriton on the public PyPI repository.[14] The attacker uploaded a malicious distribution tagged with the version string 2.0.0+0d7e753227.[6] PyTorch's official installation documentation for its nightly builds instructed users to utilize pip's --extra-index-url parameter to point to the PyTorch private repository.[6] Because the attacker intentionally assigned a highly specific and elevated version number to the public malicious package, pip evaluated all available indices, found the malicious package on PyPI, deemed it the "latest" version, and installed it by default, completely ignoring the legitimate package hosted on the private index.[5]

The malicious torchtriton package contained a compiled ELF (Executable and Linkable Format) binary named triton that was designed to execute upon installation, specifically placing itself in the PYTHON_SITE_PACKAGES/triton/runtime/ path.[5] Once triggered, the binary initiated a sweeping exfiltration of sensitive system data. It harvested the system's hostname, current username, current working directory, and environmental variables.[5] More critically, it read and copied the contents of /etc/resolv.conf, /etc/hosts, /etc/passwd, the user's .gitconfig, and up to 1,000 files located within the user's .ssh/ directory, compromising private cryptographic keys.[5]

To bypass traditional network egress filters and data loss prevention (DLP) systems, the malware did not use standard HTTP/HTTPS POST requests. Instead, it utilized DNS tunneling. The stolen files were encoded and exfiltrated via encrypted DNS queries to the domain .h4ck[.]cfd, utilizing the specific DNS server wheezy[.]io.[5]

For the PyPI-SCADA simulator, torchtriton represents a critical edge case in differential analysis. There was no Last Known Good Release (LKGR) on the public PyPI repository, as the package name had never been legitimately registered there before the attack.[13] Detection of dependency confusion cannot rely on intra-package diffing; it must rely on detecting namespace squatting combined with sudden, high-risk behavior (such as executing an ELF binary or initiating DNS queries) in a newly registered package that shares a name with known private dependencies. Following the attack's discovery on December 30, 2022, PyTorch mitigated the issue by renaming their legitimate internal package to pytorch-triton and claiming a dummy torchtriton package on PyPI to prevent future squatting.[14]

5.1.2 The totallysafe Artifact

The totallysafe package is a foundational historical artifact archived within the Backstabber's

Knife Collection.[11] Uploaded during the 2015-2019 window analyzed by Ohm et al., it serves as an early proof-of-concept for hidden execution within the Python ecosystem.[9] While exact version metrics are obscured by its early removal and subsequent academic archival, its purpose in the dataset is to demonstrate how malicious actors embed secondary payloads within standard Python setup scripts. In a SCADA simulation environment, totallysafe is utilized as a baseline control to test whether the diffing engine and SAST adapters can identify hidden arbitrary code execution buried beneath layers of seemingly benign string manipulation and obfuscation, regardless of the package's specific version history.

5.2 Account Takeover and Fruit Poisoning

"Fruit Poisoning" represents an escalation in attacker sophistication. Rather than tricking users into downloading a typographical error or exploiting dependency resolution quirks, the attacker gains unauthorized access to a highly trusted, widely utilized package that already exists within thousands of enterprise dependency trees. By poisoning the legitimate supply chain directly, the attacker leverages the established trust of the package. This is typically achieved via Account Takeover (ATO) leveraging highly targeted phishing, credential stuffing, or the exploitation of unrevoked API tokens.

5.2.1 The num2words Phishing Campaign

The num2words library is a highly popular, mature utility used to convert numerical digits into text words across multiple languages, serving as a dependency for numerous text-to-speech and data processing applications.[17] On July 28, 2025, the global Python security community was alerted to a severe supply chain compromise involving this foundational package.[8] Threat actors linked to the "Scavenger" group (also known as JuiceLedger in previous campaigns) executed a sophisticated forward-proxy phishing attack against PyPI maintainers.[8]

Standard Multi-Factor Authentication (MFA) is often considered the silver bullet for preventing account takeovers. However, the attackers bypassed this by hosting a deceptive domain equipped with an SSL certificate that transparently proxied requests to the real pypi.org.[20] The attackers scraped public GitHub repositories and package metadata to identify the email addresses of package maintainers, sending them urgent emails requesting they validate their PyPI credentials. When the maintainers clicked the link and logged in, the proxy server intercepted not only their usernames and passwords but also the live MFA session tokens, granting the attackers complete, authenticated access to the legitimate PyPI accounts.[20]

Correction of Historical Data: Initial queries hypothesized that version 0.5.13 was the malicious release. However, exhaustive forensic history confirms that 0.5.13 was a benign release published on October 18, 2023, featuring standard bug fixes for Brazilian Portuguese and Norwegian language support.[17] The actual malicious packages injected by the threat actors were versions 0.5.15 and 0.5.16, uploaded directly to PyPI on July 28, 2025.[8] The Last Known Good Release (LKGR) immediately preceding the compromise was 0.5.14, which had been legitimately published on December 17, 2024.[17]

Upon gaining access, the attackers uploaded the malicious versions to PyPI. The malware mechanics involved modifying the package's core initialization script (__init__.py) to silently

load and execute a malicious Dynamic Link Library (DLL) hidden within an obfuscated _build.py file.[21] Because the package was already implicitly trusted by the ecosystem, automated dependency management tools immediately detected the new version and began automatically opening pull requests to upgrade downstream enterprise projects to the poisoned 0.5.15 version, rapidly accelerating the infection vector.[8]

This incident provides the perfect calibration scenario for static differential analysis. The PyPI-SCADA engine, upon comparing the LKGR (0.5.14) to the poisoned update (0.5.15), would instantly detect the anomalous inclusion of a binary DLL loader and the execution of arbitrary external files in the initialization sequence. Furthermore, ecosystem metadata diffing would identify a severe discrepancy: version 0.5.15 was published to PyPI without a corresponding tag or commit history in the official source GitHub repository, a massive red flag indicating that the standard CI/CD pipeline was entirely bypassed by the attacker.[8]

5.2.2 The ultralytics CI/CD Pipeline Hijack

Ultralytics is a premier artificial intelligence and computer vision library, famously maintaining the YOLO (You Only Look Once) object detection models, boasting nearly 60 million global downloads and serving as a critical dependency in platforms like the ComfyUI Impact Pack.[22] In early December 2024, the project suffered a catastrophic supply chain compromise. Uniquely, this attack was not the result of a stolen PyPI password or a phishing attack against a maintainer. Instead, the attackers compromised the project's automated GitHub Actions CI/CD workflow.[25]

The attackers exploited a known script injection vulnerability related to the pull_request_target trigger in GitHub Actions. In GitHub CI/CD architecture, the pull_request_target trigger is inherently dangerous because it executes the workflow in the context of the base repository, rather than the isolated fork.[26] This grants the workflow privileged access to repository secrets, including the PyPI API publishing tokens required by PyPA's Trusted Publishing mechanisms.[25] By submitting a maliciously crafted pull request originating from a suspicious account named openimbot, the threat actors executed arbitrary code directly within the GitHub build environment.[27] This allowed them to inject malware into the release artifacts during the compilation phase, right before the artifacts were signed and pushed to PyPI.[25]

Correction of Historical Data: Initial assumptions pointed to version 8.0.100 as the malicious release. Forensic analysis confirms that the malicious code was injected into versions 8.3.41 and 8.3.42, which were published on December 4, 2024.[22] The LKGR prior to this attack was 8.3.40. The malicious versions contained embedded downloader code that, upon package installation via PyPI, fetched and executed the XMRig cryptocurrency miner on the victim's hardware.[22] The attack caused a massive spike in global CPU usage as downstream AI applications automatically pulled the poisoned computer vision library.[27]

For PyPI-SCADA detection engineering, this attack represents a terrifying evolution in supply chain tactics. Because the malicious code was injected during the ephemeral GitHub Actions build phase rather than committed to the permanent source code repository, the source distribution and the built wheel hosted on PyPI differed fundamentally from the public GitHub

source code.[25] A static differential analyzer cross-referencing the PyPI artifact against the linked source repository commit would immediately flag this cryptographic discrepancy, highlighting the injection of the cryptominer downloader that existed in the distributed package but not in the source code.

5.3 Multi-stage Execution and Remote Access Trojans (RATs)

Modern PyPI malware frequently utilizes multi-stage execution architecture. To evade primary detection heuristics and keep the package size small, the initial package uploaded to PyPI contains only a lightweight "dropper" or stager script. Once installed, this stager reaches out to a remote Command and Control (C2) server to download the actual, heavy-duty payload directly into system memory, bypassing static disk scans and traditional antivirus signatures.

5.3.1 The CondeTGAPIS Campaign: secmeasure and sisaws

In August 2025, cybersecurity researchers from Zscaler ThreatLabz uncovered a targeted malware campaign orchestrated by a threat actor using the handle "CondeTGAPIS".[28] This actor uploaded multiple interconnected packages to PyPI, most notably secmeasure and sisaws, designed specifically to deliver the "SilentSync" Remote Access Trojan (RAT) to Windows systems.[29]

The threat actor employed varied social engineering lures. The sisaws package (malicious version 2.1.6 uploaded August 4, 2025) utilized typosquatting, intentionally mimicking the legitimate sisa package, which is associated with Argentina's national health information system (Sistema Integrado de Información Sanitaria Argentino).[28] Conversely, the secmeasure package (malicious versions 0.1.0 through 0.1.2 uploaded August 3-4, 2025) presented itself as a legitimate cybersecurity utility, falsely claiming to be a library for "cleaning strings and applying security measures".[29]

Despite their different lures, both packages shared identical malicious behavior. Buried within their initialization scripts, the packages contained a function ironically named sanitize_input. When invoked, this function executed a heavily obfuscated, hex-encoded curl command.[29] This command reached out to a C2 framework hosted on GitHub Codespaces. By leveraging legitimate GitHub infrastructure for their C2, the attackers bypassed standard enterprise firewalls and domain reputation filters.[28] The command downloaded the SilentSync RAT, a sophisticated Python-based malware capable of complete system compromise. SilentSync enabled remote command execution, live screen capturing, and systematic exfiltration of passwords, auto-fill data, and session cookies from major browsers including Chrome, Edge, and Brave.[29]

5.3.2 The termncolor Persistence Attack

Operating in a similar multi-stage paradigm, the termncolor package combined typosquatting with advanced evasion techniques.[28] Masquerading as the highly popular text-formatting tool termcolor, termncolor (version 1.0.0, published around September 18, 2025) was downloaded nearly a thousand times before detection.[28]

Unlike simpler malware, termncolor did not contain the final payload. Instead, upon execution, it

was designed to quietly import a secondary rogue package called colorinal.[28] This initiated a complex infection chain resulting in DLL side-loading. By side-loading a rogue DLL into a legitimate, trusted Windows process, the malware established deep persistence on the victim's machine and facilitated encrypted command-and-control communications, ultimately resulting in full remote code execution (RCE).[28]

For the PyPI-SCADA simulator, multi-stage attacks require specialized detection logic. Because secmeasure, sisaws, and termncolor had no benign predecessors, LKGR metrics are not applicable; their entire package histories are inherently malicious. The SCADA system must prioritize analyzing the Abstract Syntax Trees of setup.py and __init__.py for encoded strings (like hex or Base64 variables) and unauthorized shell process invocations (such as subprocess.run or os.system executing curl/wget commands).

5.4 Typosquatting and Combosquatting

Typosquatting targets the inherent human element of software engineering. Attackers register package names that are slight typographical errors of massively popular libraries (e.g., requesys instead of requests). Combosquatting is a variation where attackers reorder words or add arbitrary prefixes/suffixes to legitimate names (e.g., python-requests).

5.4.1 The colourama Cryptocurrency Stealer

The colorama library is a cornerstone of the Python ecosystem, consistently ranking in the top 50 most downloaded packages globally, utilized by developers to add ANSI color styling to terminal outputs.[1] Taking advantage of global spelling variations between American and British English, an attacker uploaded the package colourama (version 0.1.6) to PyPI.[3]

Discovered around October 2018, colourama serves as an example of an early, crude, yet highly effective supply chain attack.[31] Unlike modern multi-stage RATs that establish complex C2 persistence, colourama was a direct financial exploit.[3] Upon installation, it utilized simple Base64 encoding to obfuscate a payload that actively searched the local operating system for cryptocurrency wallets. Furthermore, it monitored the user's clipboard data; if the user copied a string matching the regex format of a cryptocurrency wallet address, the malware replaced it with the attacker's address, hijacking the transaction and funneling funds directly to the threat actor.[3]

5.4.2 The nmap-python Combosquat

Similarly, the nmap-python package (version 1.0.0) recorded in the Backstabber's Knife Collection is a classic example of combosquatting.[34] The legitimate, widely used Python wrapper for the Nmap network security scanner is named python-nmap.[34] By simply reversing the wording around the hyphen, the attacker created a highly plausible alternative.[33]

Developers typing from memory, relying on autocomplete scripts, or referencing outdated documentation are highly susceptible to running pip install nmap-python, immediately compromising their host machines with the embedded Trojan payload without ever realizing they mistyped the package name.

6. Strategic Implications for Detection Engineering

The comprehensive forensic reconstruction of these nine packages reveals several underlying trends and second-order implications that must inform the design architecture of the PyPI-SCADA system.

6.1 The CI/CD Pipeline as the Primary Attack Surface

A critical paradigm shift has occurred between the colourama era (2018) and the ultralytics compromise (2024). Historically, attackers had to steal a developer's PyPI password to poison a package. With the advent of mandatory Multi-Factor Authentication (MFA) on PyPI and the adoption of OIDC-based Trusted Publishing, the registry interface itself has become highly resilient. Consequently, attackers have pivoted upstream, targeting the build environments. The ultralytics attack demonstrates that compromising a GitHub Actions workflow allows an attacker to bypass PyPI's authentication entirely, injecting malware during the compilation of the distribution wheel. PyPI-SCADA must therefore prioritize source-to-artifact verification—comparing the code published on PyPI directly against the GitHub source repository to identify build-time injections.

6.2 The Evolution of Evasion and Obfuscation

Early typosquats like colourama relied on simple Base64 encoding to hide static strings, which are easily detected by basic SAST tools. Modern multi-stage execution malware, such as the secmeasure (SilentSync RAT) and termncolor packages, completely abstracts the malicious logic away from the PyPI artifact. By using heavily obfuscated, hex-encoded curl commands to fetch payloads at runtime from ephemeral, highly reputable infrastructure (like GitHub Codespaces), they render traditional AST and static signature scanning nearly obsolete. For the PyPI-SCADA simulator to remain effective, it must integrate advanced taint analysis capable of flagging *any* unexpected network invocation or execution of secondary shell commands within a setup script, regardless of the target domain's reputation.

7. Required Structured Output

The following table and CSV block consolidate the precise historical metrics required for the PyPI-SCADA simulation environment, reflecting the critical corrections discovered during forensic analysis (specifically, correcting the num2words and ultralytics malicious version assumptions to reflect the actual historical breaches). For typosquats and fully malicious multi-stage campaigns, the LKGR is marked as "N/A," and the safe download command points to the legitimate package the malware was attempting to mimic or squat.

Section 1: Markdown Table

Attack Vector	Package Name	Malicious Version	Infection Date	LKGR	Safe Download Command

Dependency Confusion	torchtriton	2.0.0+0d7e753227	2022-12-25	N/A	pip download pytorch-triton --no-deps
Dependency Confusion	totallysafe	Unknown	2015-2019	N/A	N/A
Account Takeover	num2words	0.5.15	2025-07-28	0.5.14	pip download num2words==0.5.14 --no-deps
Account Takeover	ultralytics	8.3.41	2024-12-04	8.3.40	pip download ultralytics==8.3.40 --no-deps
Multi-stage Execution	secmeasure	0.1.0	2025-08-03	N/A	N/A
Multi-stage Execution	sisaws	2.1.6	2025-08-04	N/A	pip download sisa --no-deps
Multi-stage Execution	termncolor	1.0.0	2025-09-18	N/A	pip download termcolor==2.3.0 --no-deps
Typosquatting	colourama	0.1.6	2018-10-01	N/A	pip download colorama==0.4.6 --no-deps
Typosquatting	nmap-python	1.0.0	2015-2019	N/A	pip download python-nmap==0.7.1 --no-deps

Section 2: CSV Data

Code snippet

```
Attack_Vector,Package_Name,Malicious_Version,Infection_Date,LKGR,Safe_Download_Command
Dependency Confusion,torchtriton,2.0.0+0d7e753227,2022-12-25,N/A,pip download pytorch-triton --no-deps
Dependency Confusion,totallysafe,Unknown,2015-2019,N/A,N/A
Account Takeover,num2words,0.5.15,2025-07-28,0.5.14,pip download num2words==0.5.14 --no-deps
Account Takeover,ultralytics,8.3.41,2024-12-04,8.3.40,pip download ultralytics==8.3.40
```

--no-deps
Multi-stage Execution,secmeasure,0.1.0,2025-08-03,N/A,N/A
Multi-stage Execution,sisaws,2.1.6,2025-08-04,N/A,pip download sisa --no-deps
Multi-stage Execution,termncolor,1.0.0,2025-09-18,N/A,pip download termcolor==2.3.0
--no-deps
Typosquatting,colourama,0.1.6,2018-10-01,N/A,pip download colorama==0.4.6 --no-deps
Typosquatting,nmap-python,1.0.0,2015-2019,N/A,pip download python-nmap==0.7.1
--no-deps

Works cited

1. Python's Colorama Typosquatting Meets 'Fade Stealer' Malware - Imperva, accessed on March 9, 2026, https://www.imperva.com/blog/pythons-colorama-typosquatting-meets-fade-stealer-malware/
2. Hunting for Malicious Packages on PyPI : r/netsec - Reddit, accessed on March 9, 2026, https://www.reddit.com/r/netsec/comments/jsvypn/hunting_for_malicious_packages_on_pypi/
3. arXiv:2005.09535v1 [cs.CR] 19 May 2020, accessed on March 9, 2026, https://arxiv.org/pdf/2005.09535
4. Attacker Targets Packages in Multiple Programming Languages in Recent Software Supply Chain Attacks | by Aviad Gershon | Checkmarx Zero, accessed on March 9, 2026, https://zero.checkmarx.com/attacker-targets-packages-in-multiple-programming-languages-in-recent-software-supply-chain-attacks-ccb4cc7601ed
5. Malicious PyTorch dependency 'torchtriton' on PyPI | Wiz Blog, accessed on March 9, 2026, https://www.wiz.io/blog/malicious-pytorch-dependency-torchtriton-on-pypi-everything-you-need-to-know
6. Compromised PyTorch-nightly dependency chain between December 25th - Hacker News, accessed on March 9, 2026, https://news.ycombinator.com/item?id=34202662
7. PyTorch dependency 'torchtriton' on PyPI Supply Chain Attack - SentinelOne, accessed on March 9, 2026, https://www.sentinelone.com/blog/pytorch-dependency-torchtriton-supply-chain-attack/
8. Supply Chain Security Alert: num2words PyPI Package Shows Signs of Compromise, accessed on March 9, 2026, https://www.stepsecurity.io/blog/supply-chain-security-alert-num2words-pypi-package-shows-signs-of-compromise
9. Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks, accessed on March 9, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC7338168/
10. ecosyste-ms/typosquatting-dataset - GitHub, accessed on March 9, 2026,

https://github.com/ecosyste-ms/typosquatting-dataset
11. Backstabber's Knife Collection Dataset - GitHub Pages, accessed on March 9, 2026, https://dasfreak.github.io/Backstabbers-Knife-Collection/
12. Supply chain attacks in open source projects - Lund University Publications, accessed on March 9, 2026, https://lup.lub.lu.se/student-papers/record/9105730/file/9105731.pdf
13. PyTorch discloses malicious dependency chain compromise over holidays - Reddit, accessed on March 9, 2026, https://www.reddit.com/r/programming/comments/10191me/pytorch_discloses_malicious_dependency_chain/
14. Compromised PyTorch-nightly dependency chain between December 25th and December 30th, 2022., accessed on March 9, 2026, https://pytorch.org/blog/compromised-nightly-dependency/
15. PyTorch supply chain attack: Dependency confusion burns DevOps | ReversingLabs, accessed on March 9, 2026, https://www.reversinglabs.com/blog/pytorch-supply-chain-attack-dependency-confusion-burns-devops
16. PyTorch Identifies Malicious Dependency in its Nightly Build - Bitdefender, accessed on March 9, 2026, https://www.bitdefender.com/en-us/blog/hotforsecurity/pytorch-identifies-malicious-dependency-in-its-nightly-build
17. num2words - PyPI, accessed on March 9, 2026, https://pypi.org/project/num2words/
18. num2words - PyPI, accessed on March 9, 2026, https://pypi.org/project/num2words/0.5.1/
19. First Known Phishing Attack Against PyPi Users - Checkmarx, accessed on March 9, 2026, https://checkmarx.com/blog/first-known-phishing-attack-against-pypi-users/
20. PyPI Phishing Attack: Incident Report - The Python Package Index Blog, accessed on March 9, 2026, https://blog.pypi.org/posts/2025-07-31-incident-report-phishing-attack/
21. Embedded Malicious Code in num2words - Snyk Vulnerability Database, accessed on March 9, 2026, https://security.snyk.io/vuln/SNYK-PYTHON-NUM2WORDS-11172937
22. Ultralytics AI Library Hacked via GitHub for Cryptomining | Wiz Blog, accessed on March 9, 2026, https://www.wiz.io/blog/ultralytics-ai-library-hacked-via-github-for-cryptomining
23. ultralytics · PyPI, accessed on March 9, 2026, https://pypi.org/project/ultralytics/
24. Compromised ultralytics PyPI package delivers crypto coinminer - ReversingLabs, accessed on March 9, 2026, https://www.reversinglabs.com/blog/compromised-ultralytics-pypi-package-delivers-crypto-coinminer
25. Supply-chain attack analysis: Ultralytics - The Python Package Index Blog, accessed on March 9, 2026, https://blog.pypi.org/posts/2024-12-11-ultralytics-attack-analysis/

26. The Ultralytics Supply Chain Attack: How It Happened, How to Prevent - Legit Security, accessed on March 9, 2026, https://www.legitsecurity.com/blog/the-ultralytics-supply-chain-attack-how-it-happened-how-to-prevent
27. Ultralytics AI Library Compromised: Cryptocurrency Miner Found in PyPI Versions, accessed on March 9, 2026, https://thehackernews.com/2024/12/ultralytics-ai-library-compromised.html
28. Python — Latest News, Reports & Analysis | The Hacker News, accessed on March 9, 2026, https://thehackernews.com/search/label/Python
29. Malicious PyPI Packages Deliver SilentSync RAT | ThreatLabz - Zscaler, accessed on March 9, 2026, https://www.zscaler.com/blogs/security-research/malicious-pypi-packages-deliver-silentsync-rat
30. Malicious PyPI and npm Packages Discovered Exploiting Dependencies in Supply Chain Attacks - The Hacker News, accessed on March 9, 2026, https://thehackernews.com/2025/08/malicious-pypi-and-npm-packages.html
31. Detection of Intrusions and Malware, and Vulnerability Assessment: 17th International Conference, DIMVA 2020, Lisbon, Portugal, June 24–26, 2020, Proceedings [1st ed.] 9783030526825, 9783030526832 - DOKUMEN.PUB, accessed on March 9, 2026, https://dokumen.pub/detection-of-intrusions-and-malware-and-vulnerability-assessment-17th-international-conference-dimva-2020-lisbon-portugal-june-2426-2020-proceedings-1st-ed-9783030526825-9783030526832.html
32. The installation of a Python package from PyPi could have infected the machine of your Windows users with malware : r/sysadmin - Reddit, accessed on March 9, 2026, https://www.reddit.com/r/sysadmin/comments/9sxkhb/the_installation_of_a_python_package_from_pypi/
33. A Machine Learning-Based Approach For Detecting Malicious PyPI Packages - arXiv, accessed on March 9, 2026, https://arxiv.org/html/2412.05259v1
34. Beyond Typosquatting: An In-depth Look at Package Confusion - Lorenzo De Carli, accessed on March 9, 2026, https://ldklab.github.io/assets/papers/usenix23-confusion.pdf
35. Practical IoT Hacking: The Definitive Guide to Attacking the Internet of Things - Directory, accessed on March 9, 2026, https://files.nochill.in/books/Fotios%20Chantzis,%20Ioannis%20Stais,%20Paulino%20Calderon,%20Evangelos%20Deirmentzoglou,%20Beau%20Woods%20-%20Practical%20IoT%20Hacking_%20The%20Definitive%20Guide%20to%20Attacking%20the%20Internet%20of%20Things-No%20Starch%20Press%20(2021).pdf
36. https://t.me/bookzillaaa - https://t.me/ThDrksdHckr, accessed on March 9, 2026, http://103.203.175.90:81/fdScript/RootOfEBooks/E%20Book%20collection%20-%202025%20-%20A/CSE%20%20IT%20AIDS%20ML/Practical_IoT_Hacking_by_Fotios_Chantzis,_Ioannis_Stais,_Paulino.pdf